Mattia Loi

Midterm Project

Tasks:

1)

In Task 1, we had to load the DataFrame from the file *"data.csv"*. We can see that it matches the description provided in the project statement, with its various columns and values. The dataset has a shape of (3090x14), and the first thing I decided to do was to remove any possible duplicates from the DataFrame.

2)

In Task 2, I didn't follow the exact order provided in the instructions but completed every point. I started by identifying the 'NaN' values, which are invalid in our DataFrame. After noticing a few explicit NaN entries, I decided to explore the dataset further to understand how the values were distributed across the different features.

I found that some features contained invalid values that could be treated as NaN, while others (like *work_sector*) had strange or inconsistent values, which I decided to drop. For all other features with unacceptable values, I chose to either reassign new valid ones or set them as NaN.

After plotting the data, I observed that some features contained negative values; since they were few, I dropped them along with all records having an *age* lower than 18, which showed suspicious values.

After this first cleaning phase, only a few NaN values remained in some features, so I decided to encode all categorical features and then use the KNN algorithm to impute the missing values of *smoker* and *physical_activity*, which had a significant amount of missing data and were important for further analysis.

I encoded the variables using different methods — *one-hot*, *binary*, and *cardinal encoding depending* on the case. I applied *outlier capping* to ensure that KNN (which is sensitive to outliers could perform better and to keep the data instead of removing outliers completely.

I also applied a *log transformation* to the *income* feature, which was right-skewed; after the transformation, it showed a normal distribution. This is important because *income* strongly influences our target, and models tend to perform better with normally distributed values.

Finally, I applied *StandardScaler* before running KNN (since outliers had already been handled) with $k = 3$, which allowed me to infer the missing values for *smoker* and *physical_activity*.

At the end of Task 2, I obtained a clean DataFrame with no duplicates, no outliers, no NaN values, normally distributed numeric features, and all categorical features properly encoded — ready to be used in the following models.

3)

The dataset obtained from Task 2 is now complete. I decided to perform *feature selection* using several methods ANOVA*, Decision Tree-based models, Linear Models,* and *Recursive Feature Elimination (RFE)*. I selected the best-performing features from each approach, obtaining a *reduced DataFrame* with 17 features roughly half of the original ones.

From this reduced DataFrame, I created two versions: one including the *income (USD)* feature and one without it. I then applied *StandardScaler* to the numeric variables (excluding the target variable).

A) In point 3a, I used *Linear Regression* (based on the sklearn implementation) on both reduced datasets with and without *income (USD)* obtaining the following results:

- With *income*: **R^2 Score = 0.7596**, **MSE = 34,302,279.3806**
- Without *income*: **R^2 Score = 0.6496**, **MSE = 49,985,216.0700**

As we can see, the model performs worse without the *income* feature, although the results are still acceptable.

B)

In point 3b, I developed a *Linear Regression* model using *Batch Gradient Descent*. I carefully handled potential NaN-related issues that could appear with certain learning rates, and I added an early stopping condition once the model reached convergence to avoid unnecessary computations.

First DataFrame (with income):

- **R^2 Score:** 0.7603
- **MSE:** 34,192,182.2152
- **Optimal Learning Rate:** 0.96
- **Final Iterations:** 821

Second DataFrame (without income):

- **R^2 Score:** 0.6502
- **MSE:** 49,899,154.5687
- **Optimal Learning Rate:** 0.11
- **Final Iterations:** 2000

C) In point 3c, I developed a *Linear Regression* model using *Mini-Batch Gradient Descent*, testing batch sizes as powers of two up to 128.

For the first DataFrame (with income):

- **Optimal Batch Size:** 64
- **R^2 Score:** 0.7604
- **MSE:** 34,181,447.3731
- **Optimal Learning Rate:** 0.01

For the second DataFrame (without income):

- **Optimal Batch Size:** 128
- **R^2 Score:** 0.6502
- **MSE:** 49,900,798.5621
- **Optimal Learning Rate:** 0.01
- **Final Iterations:** 2000

As we can see, performance decreases without the *income* feature, though results remain acceptable.

D) In point 3d, we were asked to build an *augmented DataFrame* to capture nonlinear correlations by combining features. I decided not to use a built-in function but to manually construct the augmented dataset based on my own feature-selection criteria.

I then ran a *Gradient Descent* algorithm on this new dataset. Due to some issues, I had to modify the *Batch Gradient Descent* code, as it was returning "NaN" inputs (despite data being clean). This was likely caused by some numerical computations, so I added a validation check.

Final Model Performance:

- **R^2 Score:** 0.8544
- **MSE:** 20,768,742.6409
- **Optimal Learning Rate:** 0.081
- **Final Iterations:** 2000

These results are significantly better than those obtained with the reduced DataFrame.

F) In point 3f, we were asked to train a *Linear Regression model with Lasso regularization* and comment on the variation in the number of features as *alpha* changes. I selected widely spaced alpha values, and as expected, increasing alpha reduced the number of selected features. The best result was:

- **Highest R^2 Score:** 0.8673
- **Corresponding Alpha:** 5
- **Number of Features:** 191

G) Using the augmented dataset again, I trained a *Ridge Regression* model with the same alpha values used in Lasso. As expected, the number of coefficients remained constant, since Ridge doesn't eliminate features but assigns them very small (near-zero) weights. The results were:

- **Highest R^2 Score:** 0.8621
- **Corresponding Alpha:** 1
- **Number of Features Effectively Used:** 274

Results are comparable to Lasso, with a solid R^2 score.

4) In Task 4, we were asked to *stratify the dataset by gender*, producing four total DataFrames — one for each gender and for the presence or absence of the *income (USD)* feature. I then trained the *Batch Gradient Descent* model on each dataset and obtained the following results:

Male:

- *With income (USD)* → **R^2 Score:** 0.7193, **MSE:** 43,055,054.8622, **Learning Rate:** 0.026
- *Without income (USD)* → **R^2 Score:** 0.6453, **MSE:** 54,413,550.6133, **Learning Rate:** 0.026

Female:

- *With income (USD)* → **R^2 Score:** 0.7480, **MSE:** 33,888,380.1719, **Learning Rate:** 0.017
- *Without income (USD)* → **R^2 Score:** 0.6875, **MSE:** 42,017,223.9344, **Learning Rate:** 0.026

As we can observe, the absence of the *income* feature once again worsens the model's performance, and overall, the female model performs slightly better in both cases.

5)

Task / Model	Dataset	R^2 Score	MSE
3a - Linear Regression (sklearn)	With income	0.7596	34302279.38
3a - Linear Regression (sklearn)	Without income	0.6496	49985216.07
3b - Linear Regression (Batch GD)	With income	0.7603	34192182.22
3b - Linear Regression (Batch GD)	Without income	0.6502	49899154.57
3c - Linear Regression (Mini-Batch GD)	With income	0.7604	34181447.37
3c - Linear Regression (Mini-Batch GD)	Without income	0.6502	49900798.56
3d - Gradient Descent on Augmented DataFrame	Augmented	**0.8544**	20768742.64
3f - Lasso Regression	Augmented	**0.8673**	-
3g - Ridge Regression	Augmented	**0.8621**	-
4 - Batch GD (Male)	With income	0.7193	43055054.86
4 - Batch GD (Male)	Without income	0.6453	54413550.61

4 - Batch GD (Female)	With income	0.7480	33888380.17
4 - Batch GD (Female)	Without income	0.6875	42017223.93

Income is the most influential feature, as all models perform worse without it. Linear Regression, Batch GD, and Mini-Batch GD yield similar results, showing that the optimization method has little effect on accuracy. Augmenting the dataset with nonlinear features significantly improves performance (R^2 up to 0.8544, MSE reduced), and regularized models (Lasso and Ridge) further stabilize predictions. Stratifying by gender shows slightly better results for females, highlighting subgroup differences. Overall, feature selection and augmentation have a larger impact than the choice of linear regression methods.